1. Background

Heavy manufacturing facilities are highly energy-intensive operations. While integrating renewable energy (like solar) reduces the carbon footprint and grid dependency, a significant challenge remains: **Solar generation curves rarely match production load curves perfectly.** During peak daylight hours, excess solar energy is often wasted, while peak production times still rely heavily on expensive grid imports. There is a critical need for an intelligent system to visualize real-time energy flow and dynamically shift flexible loads to match peak solar generation.

2. Project Objective

The primary objective of **EcoFoundry 6.0** is to maximize solar energy self-consumption and minimize grid dependency by intelligently scheduling intermittent/flexible industrial tasks. The system aims to:

- Provide real-time visibility into power flow, generation, and consumption metrics.
- Forecast tomorrow's solar generation and baseline production load.
- Utilize Generative AI to automatically generate an optimized daily schedule that fits flexible tasks into forecasted "Free Energy" (solar surplus) slots.

3. Scope of Work

The project encompasses a full-stack data application with the following scope:

- **Live Energy Dashboard:** A custom-styled UI tracking daily performance, power flow curves, and energy breakdowns across factory operations.
- **Demand & Supply Forecasting:** A predictive module combining a Random Forest machine learning model with scaled historical load profiles based on active production plans.
- **AI Job Scheduler:** A planner that takes a user-defined backlog of jobs and uses an LLM (Large Language Model) to assign start/end times based strictly on available solar surplus.
- **Gantt Chart Visualization:** Dynamic, visual representation of the AI-generated schedule.

4. Future Scope

- **Automated ML Pipeline:** Currently, the predictive solar model (`solar_model.pkl`) is static. Future phases will introduce a pipeline for monthly retraining and fine-tuning of the

Random Forest model using newly ingested weather and generation data to prevent model drift.
- **User Based Login:** Once the AI schedules the task, that schedule should reach the respective person for this we can add user based login in future.

5. Technology Stack

- **Frontend UI:** Custom HTML/CSS for modern styling.
- **Backend Logic:** Python, Pandas (for heavy data manipulation, resampling, and time-series merging).
- **Database:** DuckDB (Chosen for blazing-fast analytical queries and seamless file-based deployment).
- **Machine Learning:** `scikit-learn` (Random Forest for Solar Forecasting).
- **AI / LLM Engine:** Google Gemini (via direct REST API for robust, dependency-free HTTP requests).
- **Deployment:** Render (Cloud Application Platform).

6. Key Challenges & Solutions

1. **Dynamic UI Rendering:** Dash DataTables struggled with dynamic dropdowns. *Solution:* Built a robust dictionary lookup system and enforced per-row dropdown schemas.
2. **AI Model API Volatility:** The standard Google Generative AI Python library threw environment-specific versioning errors. *Solution:* Bypassed the library by writing a direct HTTP REST API connector using `requests`, paired with robust Regex parsing to extract clean JSON.
3. **Forecasting Realism:** Calculating baseline loads using simple historical averages skewed expectations downwards due to weekends. *Solution:* Updated SQL/DuckDB queries to filter specifically for weekdays and integrated live production statuses to dynamically scale the load profile.
4. **Cloud Database Deployment:** Traditional PostgreSQL requires persistent cloud servers. *Solution:* Migrated to **DuckDB** and deployed on **Render**, allowing the application to run a high-performance analytical database locally within the deployed environment without paid server overhead.